

Chapter 1

Reinforcement Learning

This chapter introduces the fundamental concepts of reinforcement learning, providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the standard references in reinforcement learning, in particular the works of Sutton and Barto [?]. Its structure is as follows:

- **Historical developments:** an overview of the key ideas and milestones that led to the emergence of reinforcement learning as a unified field.
- **Objective:** the mathematical definition of the reinforcement learning goal through Markov Decision Processes, policies, and value functions.
- **Learning:** how policies are progressively refined through interaction with the environment to approach the optimal policy.

1.1 Historical Developments

Reinforcement learning is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

1.1.1 Behavioral foundations (1900–1950)

The origins of reinforcement learning can be traced back to 20th-century animal psychology studies. These studies aimed to understand how behaviors are formed and modified in animals. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be reinforced and repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency [?].

This perspective was later developed within behaviorism. In this school of psychology, reinforcement and punishment are used to gradually shape behavior through repeated interaction with the environment [?]. The term “reinforcement” in the context of learning comes from this school of thought. This perspective in

behaviorism influenced early thinking in computing and artificial intelligence. It led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a “pleasure–pain system” [?].

Although these studies did not involve formal mathematical models, they introduced key ideas that later became central to reinforcement learning, including trial-and-error learning and adaptation driven by environmental feedback.

1.1.2 Sequential decision-making (1950–1970)

A second key foundation of reinforcement learning comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, an agent is the decision-making entity that interacts with the environment by selecting actions. A policy is the function that maps an observed state to the action chosen by the agent. In this field, the objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDPs), which provide a mathematical model for sequential decision-making under uncertainty [?]. MDPs are defined by four main components: states, actions, rewards, and transition dynamics. A state represents the current situation of the environment at a given time. An action is a choice available to the policy in a given state. A reward is a scalar signal that provides feedback after each action. Transition dynamics describe how the environment evolves from one state to another.

One of the earliest approaches in this framework is dynamic programming [?]. This method relies on a recursive decomposition of sequential decision problems via the principle of optimality. By solving each subproblem corresponding to a state, dynamic programming computes the optimal value function and, from it, derives the optimal policy. However, classical dynamic programming methods rely on the assumption that the environment dynamics are fully known. This means that the full state space and all transition relationships must be explicitly modeled, which quickly becomes computationally intractable in realistic problems.

1.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference (TD) learning methods were introduced [?]. TD learning combines ideas from Monte Carlo methods and dynamic programming by updating predictions based on other learned estimates, rather than waiting for final outcomes.

This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations and forms a key building block of reinforcement learning algorithms.

Building on these new ideas, Q-learning was introduced, showing that optimal action-value functions can be learned directly from interaction with the environment without requiring a model of its dynamics [?, ?]. The publication of the

foundational reference work [?] contributed to the formalization of reinforcement learning as a coherent field.

1.1.4 Consolidation of reinforcement learning (2000–2013)

A key limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDPs, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced, allowing these quantities to be represented using parametric models such as linear approximators or perceptron-like architectures [?, ?]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space.

The introduction of Deep Q-Networks (DQN), which demonstrated that deep Neural Network (NN) could be used to approximate value functions at scale while maintaining stability [?]. By combining Q-learning with deep NN architectures, DQN successfully extended value-based reinforcement learning to high-dimensional input spaces such as raw pixels in Atari games.

1.1.5 Deep reinforcement learning (2013–2026)

The introduction of deep NN into reinforcement learning marked a major turning point in the field. Building on this idea, a series of algorithms significantly improved scalability. Deep Deterministic Policy Gradient (DDPG) extended actor–critic methods to continuous action spaces [?]. Proximal Policy Optimization (PPO) later became a standard algorithm in reinforcement learning due to its simplicity, stability, and ability to perform in both continuous and discrete action spaces [?].

These advances established a practical foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NN, reinforcement learning, and tree search could reach superhuman performance in the game of Go [?]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [?]. Similarly, in Dota 2, large-scale multi-agent reinforcement learning systems such as OpenAI Five showed that self-play combined with deep policy optimization can achieve professional-level performance in highly dynamic and stochastic environments [?].

More recently, reinforcement learning has expanded beyond its traditional application domains, such as robotics, video games, and combinatorial optimization. With the emergence of increasingly agentic views of large language models, it has become possible to train these models to perform specific tasks using reinforcement learning-based techniques. In particular, reinforcement learning from human feedback (RLHF) has become a key method for aligning large language models with human preferences by combining reward modeling and policy

122 optimization [?]. Similarly, DeepSeekMath shows how reinforcement learning
 123 can enhance the mathematical reasoning abilities of large language models [?]. It
 124 demonstrates that an artificial agent can learn to reason in natural language in
 125 order to accomplish a given task. The skills acquired during this learning process
 126 also transfer to other similar tasks, leading to improved overall performance.
 127 For instance, training a large language model with reinforcement learning on
 128 mathematical problem-solving can also enhance its programming capabilities.

129 Originally inspired by biological learning processes, reinforcement learning is
 130 now closer than ever to this initial intuition. Agents learn through interaction,
 131 adapt via trial and error, and progressively improve their behavior based on
 132 experience rather than explicit supervision. Combined with deep neural architec-
 133 tures, this paradigm strengthens the analogy with natural learning mechanisms,
 134 further aligning computational learning processes with those observed in living
 135 organisms. Thanks to these advances, reinforcement learning systems can learn
 136 internal representations of their environment and generalize beyond previously
 137 encountered situations. These advances contribute to the ongoing progress
 138 toward artificial general intelligence, and more than ever, reinforcement learning
 139 stands as one of the core paradigms for such systems [?].

140 1.2 Objective

141 Now that the main developments shaping reinforcement learning have been out-
 142 lined, we turn to the formal definition of its objective. In essence, reinforcement
 143 learning aims to build a system that makes sequential decisions in order to
 144 maximize a cumulative reward signal over the long term. The purpose of this
 145 section is to translate this intuitive goal into a rigorous mathematical framework.
 146 To this end, we proceed step by step:

- 147 1. We define how problems are modeled using Markov Decision Processes in
 148 Subsection 1.2.1.
- 149 2. We introduce the notion of policy, which formalizes the agent's behavior,
 150 in Subsection 1.2.2.
- 151 3. We describe how the interaction between the agent and the environment is
 152 characterized through trajectories in Subsection 1.2.3.
- 153 4. We present how policies are evaluated using value functions in Subsec-
 154 tion 1.2.4.
- 155 5. We define the optimal policy in Subsection 1.2.5, which is exactly what we
 156 seek to obtain when applying reinforcement learning.

157 1.2.1 Markov Decision Processes

158 The RL framework is grounded in the agent paradigm: the decision-making
 159 entity is called the agent. The agent perceives its environment, processes the
 160 information it receives, and acts upon the environment to influence future
 161 outcomes. To formally specify both the task and the agent's capacity to perceive
 162 and act, we model the problem as a Markov Decision Process (MDP).

Before providing a more detailed definition, it is useful to clarify the scope of the MDP problems considered in this manuscript. The goal of this subsection is to introduce the notations and formalisms as they are used in modern RL practice. Other formulations of MDPs exist that are better suited, for instance, to proving convergence guarantees. However, our focus here is to represent reinforcement learning as it is applied empirically, even if this means sacrificing some mathematical proof capability. We therefore restrict our analysis to the following configuration:

- We consider the single-agent learning setting. If any other agents are present in the environment, they are treated as part of the environment dynamics.
- We focus on finite-horizon MDPs. Each interaction sequence between the MDP and the policy has a defined beginning and a defined end. Between these two points, the number of interaction steps may vary depending on the situation, but it always remains finite.
- We adopt the partially observable framework, where the agent does not have direct access to the true state of the system, but instead relies on limited observations that provide only partial information about the environment.

The resulting framework can be formally described as a single-agent finite-horizon partially observable markov decision process. For simplicity, we will refer to this setting as an “MDP” throughout this manuscript.

Now that the scope of the considered problems has been defined, we formally introduce the model. A MDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- $\mathcal{S}$: The set of states, representing all possible configurations of the environment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among these, we distinguish two special subsets: initial states, from which an interaction can begin, and terminal states, in which the interaction ends. The set of initial states is denoted by $\mathcal{S}^i$, with an initial state written as $s^i \in \mathcal{S}^i$. The set of terminal states is denoted by $\mathcal{S}^t$, with a terminal state written as $s^t \in \mathcal{S}^t$.
- $\mathcal{O}$: The set of observations that the agent can perceive from the environment. Unlike the true state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$ provides partial information about the current state.
- $\mathcal{A}$: The set of actions that the agent may execute. Actions influence the dynamics of the MDP, thereby shaping its future course. They represent the agent’s capacity to act upon its environment.
- $\mathcal{R}$: The set of rewards that the agent can receive from the environment. $\mathcal{R}$ is a subset of $\mathbb{R}$. The agent’s goal is to maximize the cumulative sum of these rewards over time.
- p : The transition function, which defines the dynamics of the environment. Given a state s and an action a , this function defines a probability distribution over the joint outcome consisting of the next state s' , the observation o , and the reward r . We denote this function by p , and it is defined as: $p : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o, r) \sim p(s, a)$ means that

the tuple (s', o, r) is sampled from the distribution induced by $p(\cdot \mid s, a)$.
 The notation $p(s', o, r \mid s, a)$ denotes the probability of observing the tuple
 (s', o, r) under the distribution $p(\cdot \mid s, a)$.

1.2.2 Policy

The agent's decision-making capability is modeled by a policy function. The policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions $a \in \mathcal{A}$. We denote a policy function by π , and it is defined as: $\pi : \mathcal{O} \rightarrow \mathcal{P}(\mathcal{A})$. The notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution induced by $\pi(\cdot \mid o)$. The notation $\pi(a \mid o)$ denotes the probability of selecting action a under the distribution $\pi(\cdot \mid o)$.

1.2.3 Trajectory

Now that both the MDP and the policy have been defined, we can characterize the interaction between them. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment provides an observation o to the policy π . Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot \mid o)$, from which an action is sampled: $a \sim \pi(\cdot \mid o)$.
2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot \mid s, a)$. It moves to a new state s' , and returns a new observation o' and a reward r .

A trajectory t of length T can be written as:

$$t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T).$$

It is important to clearly distinguish between a trajectory distribution and its realizations. We denote by τ the distribution over trajectories, and by t a particular trajectory. The distribution τ is determined by the initial state s , the initial observation o , the environment dynamics p , and the policy π , and is denoted by $\tau(s, o, p, \pi)$. The notation $t \sim \tau(s, o, p, \pi)$ means that the trajectory t is sampled from the distribution induced by s , o , p , and π .

An important property of the trajectory distribution is its recursive structure. Since both the policy π and the transition function p are known, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2 . Specifically, the probability of a trajectory $t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T)$ factorizes as:

$$\tau(t \mid s_1, o_1, p, \pi) = \pi(a_1 \mid o_1) p(s_2, o_2, r_1 \mid s_1, a_1) \tau(t_2 \mid s_2, o_2, p, \pi),$$

There exist two special types of trajectories. First, a trajectory that terminates in a terminal state is called a terminal trajectory, and is denoted by $t_t \sim \tau_t(s, o, p, \pi)$. Second, a terminal trajectory that starts from an initial state and an initial observation is called an episode, and is denoted by $t_e \sim \tau(s^i, o, p, \pi)$.

1.2.4 Value Functions

To evaluate the ability of a policy π to collect rewards, we introduce the notion of return. The return of a trajectory $g(t, \gamma)$, is defined as the discounted sum of rewards collected along the trajectory:

$$g(t, \gamma) = r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots + \gamma^{T-1} r_T = \sum_{i=1}^T \gamma^{i-1} r_i,$$

where $\gamma \in [0, 1]$ is the discount factor. This parameter controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, we would like to set $\gamma = 1$. In practice, however, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain.

The return also admits a recursive form, which is central to the dynamic programming and reinforcement learning algorithms presented later:

$$g(t, \gamma) = r_1 + \gamma g(t_2:),$$

where $t_2: = (s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T)$ denotes the trajectory starting from the second step.

From the notion of return, we can define functions that estimate the expected return of a policy from a given starting point. These functions are called value functions.

The first one we examine in detail is the state-value function. It tells us how desirable a particular state is under a given policy π , given an observation o and environment dynamics p . Strictly speaking, this function should be written as $V(s, o, p, \pi)$ to reflect all its dependencies. However, since the environment dynamics p are fixed throughout learning, we omit it from the notation. To emphasize that the value is tied to a specific policy π , we place π as a subscript. This yields the compact notation $V_\pi(s, o)$.

If $V_\pi(s, o) > V_\pi(s', o')$, then, under policy π , the agent prefers being in state s with observation o over state s' with observation o' . Formally, the state-value function is defined as:

$$V_\pi(s, o) = \mathbb{E}[g(t)], \quad \text{with } t \sim \tau_t(s, o, p, \pi).$$

It is worth noting that the state-value function V_π is recursive, a property that follows directly from the recursive form of the return. Using $g(t) = r_1 + \gamma g(t_2:)$, we can write:

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}[g(t)], \quad \text{with } t \sim \tau_t(s, op, \pi) \\ &= \mathbb{E}[r_1 + \gamma g(t_2:)] \\ &= \mathbb{E}[r_1 + \gamma V_\pi(s_2)], \end{aligned}$$

If we express the expectation explicitly in terms of the distributions p and π , we obtain the Bellman equation for V_π :

$$\begin{aligned}
V_\pi(s, o) &= \mathbb{E} \left[r_1 + \gamma V_\pi(s_2) \right], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\
&= \sum_{a \in \mathcal{A}} \pi(a \mid o) \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r \mid s, a) \left[r + \gamma V_\pi(s', o') \right].
\end{aligned}$$

Now that the value of a state has been expressed, the next step is to consider the value of taking a specific action in a given state. Indeed, it is useful to estimate the expected future gain of an action a in state s , in order to compare different actions and determine which one is the most promising under a given policy π . This function is called the state-action value function Q_π , and it estimates the expected return when starting from a state s , taking an action a , and then following the policy π for all subsequent steps. If $Q_\pi(s, a) > Q_\pi(s, a')$, then, under policy π , choosing action a in state s yields a higher expected return than choosing action a' . It is defined as:

$$Q_\pi(s, a) = \mathbb{E} \left[r + \gamma g(t) \right], \quad \text{with } t \sim \tau_t(s', o, p, \pi), \quad (s', o, r) \sim p(\cdot \mid s, a).$$

We can relate Q_π to V_π by recognizing that, after taking action a in state s , the remaining return from the next state s' is precisely $V_\pi(s')$. This yields:

$$\begin{aligned}
Q_\pi(s, a) &= \mathbb{E} \left[r + \gamma g(t) \right], \quad \text{with } t \sim \tau_t(s', o, p, \pi), \quad (s', o, r) \sim p(\cdot \mid s, a) \\
&= \mathbb{E} \left[r + \gamma V_\pi(s', o) \right] \\
&= \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r \mid s, a) \left[r + \gamma V_\pi(s', o) \right]
\end{aligned}$$

1.2.5 Optimal Policy

Recall that the goal of reinforcement learning is to find a policy that maximizes, for every observation, the expected cumulative reward until a terminal state is reached. This policy is called the optimal policy and is denoted by π^* . Formally, π^* is a deterministic policy that always selects the action with the highest state-action value:

$$\pi^*(a \mid o) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} Q_{\pi^*}(s, a'), \\ 0 & \text{otherwise,} \end{cases}$$

where s is the true state underlying the observation o .

As we have just seen, both V_π and Q_π can be expressed explicitly in terms of the environment dynamics p and the policy π . If p and π are fully known, one can theoretically compute the optimal policy by unfolding the graph of all possible trajectories starting from a given state s and observation o . This is possible precisely because V_π and Q_π are recursive functions. Solving an MDP in this way is known as dynamic programming.

In practice, most environments do have too many states to allow an explicit representation of the dynamics p . Storing and computing over the full transition

function becomes intractable. To overcome this limitation, instead of directly constructing the optimal policy, we adopt an iterative approach: starting from an initial policy, we progressively improve it so that it converges toward π^* . This is the subject of the next section.

1.3 Learning

Now that the optimal policy π^* has been formally defined, we face the practical question of how to obtain it. The recursive Bellman equations for V_π and Q_π suggest dynamic programming approaches, but these scale poorly to large state spaces. Reinforcement learning offers an alternative: it seeks to solve the MDP iteratively, by generating a sequence of policies that progressively approach π^* through direct interaction with the environment. In this section, we introduce two major families of reinforcement learning methods used today: critic-only methods and actor-critic methods.

1.3.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function Q_π , which we denote by $\hat{Q}_\pi$. Although $\hat{Q}_\pi$ is only an approximation of the true Q_π , the policy always acts greedily with respect to this approximation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} \hat{Q}_\pi(s, o, a'), \\ 0 & \text{otherwise.} \end{cases}$$

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from scanning over the set of actions to maximize $\hat{Q}_\pi$. In other words, the policy has no independent existence of its own.

To give the reader a clear example of this family of methods, we present it in its simplest form: model-free, without temporal-difference, replay buffer, bootstrapping, and using only ε -greedy exploration. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

Any parametric model can be used for $\hat{Q}$, as long as we have a rule to correct its predictions based on observed errors. Since $\hat{Q}$ aims to predict a scalar value in $\mathbb{R}$, training it is a supervised regression task. Any machine learning model capable of handling regression can therefore be used to represent $\hat{Q}$. We denote the set of parameters of the model by θ . In deep reinforcement learning implementations, a neural network is used to model this function, in which case θ represents the weights and biases of the network.

In this setting, the action selection rule is modified as follows. With probability $1 - \varepsilon$, the policy selects the greedy action that maximizes $\hat{Q}_\pi$, and with probability ε it selects an action uniformly at random. The resulting policy, which now depends on both π and ε , induces a trajectory distribution denoted by $t \sim \tau(s, o, p, \pi, \varepsilon)$.

The parameter ε controls the degree of exploration: if the policy always chooses the action it currently believes to be optimal, it may miss better alternatives that $\hat{Q}_\pi$ has not yet learned to value correctly. Exploring suboptimal actions

from time to time is therefore essential to verify that $\hat{Q}_\pi$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given starting state s_1^i and observation o_1 , the interaction produces a episode t_e sampled from $\tau_e(s_1, o_1, p, \pi, \varepsilon)$. For each step i along the episode, the return from that step onward is $g(t_{i:}) = \sum_{k=i}^T \gamma^{k-i} r_k$, which serves as the target value for the state-action pair (s_i, a_i) .
2. **Model update.** The tuples $(s_i, a_i, g(t_{i:}))$ obtained in the previous step serve as supervised training data: the state-action pair (s_i, a_i) is the input x , and the return $g(t_{i:})$ is the associated target. These examples are used to train $\hat{Q}_\theta$. We denote the improved approximation by $\hat{Q}'_\theta$, which now predicts returns more accurately under π . Since $\hat{Q}'_\theta$ has changed, the greedy policy derived from it also changes. we denote this new policy by π' .

Because the policy is now π' , the approximation $\hat{Q}'_\theta$ must be trained to predict returns under π' . This requires returning to step 1 to collect fresh interaction data generated by π' . The two steps thus alternate until the policy and the value approximation stabilize.

Even this simple algorithm already illustrates a difficulty inherent to all forms of reinforcement learning: the instability of the learning process. Each update from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ causes the policy to shift from π to π' . For the learning to remain stable, the change from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ should be as small as possible. This way, $\hat{Q}'_\theta$, trained to predict returns under π , remains reasonably accurate under π' . However, the smaller the change, the slower the learning progresses. A trade-off must therefore be struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop.

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time. In modern implementations, the value function is approximated by a neural network, which offers far better generalization and makes the method applicable to large-scale. Although theoretical convergence proofs no longer hold in this approximate setting, deep critic-only methods have demonstrated remarkable empirical success, making them the tool of choice for complex RL tasks.

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must scan over all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of reinforcement learning algorithms exists: actor-critic methods.

1.3.2 Actor-Critic Methods

Two principal modifications distinguish actor-critic methods from critic-only approaches. First, instead of approximating the state-action value function Q , we now aim to approximate the state-value function V . Second, the policy π is given its own dedicated set of parameters. The policy must output, for a given observation, a probability distribution over the action space $\mathcal{A}$. We thus have two separate components: a value function approximation $\hat{V}_{\theta_1}$ and a parameterized policy π_{θ_2} .

Analogously to critic-only methods, $\hat{V}_{\theta_1}$ is trained using supervised learning on pairs obtained from interaction with the MDP. For each step i along a trajectory, the input is the state-observation pair (s_i, o_i) and the target is the observed return $g(t_i)$. The value function is thus trained to correct its predictions based on data collected by the current policy interacting with the environment.

The key difference lies in how the policy π itself is optimized. Since the policy now has its own set of parameters θ_2 , it can be updated directly via gradient ascent on the expected return. The update uses the notion of advantage: when a taken action a in state s yields a return $g(t)$ that exceeds the predicted value $\hat{V}_{\theta_1}(s, o)$, the advantage $A(s, a) = g(t) - \hat{V}_{\theta_1}(s, o)$ is positive, and the policy parameters θ_2 are adjusted to increase the probability $\pi_{\theta_2}(a | o)$. Conversely, if the return falls short, the advantage is negative and the probability is decreased. By using $\hat{V}_{\theta_1}$ as a baseline, the updates enjoy lower variance without introducing bias, which stabilizes learning.

Both $\hat{V}_{\theta_1}$ and π_{θ_2} are improved simultaneously within the same loop: the critic provides a learned baseline for the actor, and the actor generates the data used to train the critic. This interplay reduces the instability inherent to reinforcement learning, although it does not fully eliminate it.

It is within this family of methods that the currently most widely used algorithm resides: PPO (Proximal Policy Optimization), which has become the standard and will be used throughout this manuscript for all subsequent reinforcement learning experiments. Now that we are capable of training highly performant policies, the question naturally arises of how to explain their behavior. This is the subject of the next chapter: explainability in the context of reinforcement learning.